Technical Code Review: Action-Oriented Explainable AI Framework

Submitted to: Dr. K. Namitha
Submitted by: K.lahari
Research area: Explainable AI, Causal Machine Learning, Counterfactual Explainability
Date: 3 November 2026

Github link: **https://github.com/Kuncha-Lahari/Action-Oriented-XAI-Framework**

1. Introduction

This document explains the implementation of the Action-Oriented XAI Framework and supplements the project demonstration video. It covers each module, key implementation decisions and current limitations.

Conventional classifiers output only a class or probability, and standard counterfactual methods can suggest changes that are unrealistic or not actionable. The framework addresses this by combining:

1. Predictive modelling and model selection
2. PC Algorithm-based causal structure discovery and SCM representation
3. DiCE-based counterfactual generation
4. Causal-eligibility and domain-consistency validation
5. Semantic evidence retrieval (embeddings, resource index, optional O*NET)
6. LLM-based action-plan synthesis with grounding validation
7. Multi-case evaluation and baseline comparison
8. A Streamlit application

The implementation uses the UCI Adult Income dataset; the target is whether annual income exceeds $50,000.

2. System Architecture

Input profile → Prediction → Causal discovery → Counterfactual generation → Constraint validation → Evidence retrieval → Action plan → Evaluation

Module	File	Responsibility
1	loader.py	Dataset loading, cleaning, preprocessing
2	predictor.py	Model training, comparison, selection
3	causal_scm.py	PC Algorithm and SCM construction
4	counterfactual.py	DiCE generation, validation, ranking
5	rag_retrieval.py	Semantic retrieval and O*NET integration
6	llm_synthesis.py	Evidence-grounded action-plan generation

Module	File	Responsibility
7	evaluator.py	Evaluation and baseline comparison
8	main.py	Pipeline orchestration
UI	app.py	Manual input and visualization
Support	baselines.py	Alternative counterfactual baselines
Config	config.yaml / config.py	Central parameter management

3. Module 1: Data Loading and Preprocessing (loader.py)

The loader downloads the UCI Adult dataset (a synthetic fallback is used if the download fails). Categorical variables are encoded with LabelEncoder, numerical variables are cast to float, and the original `education` column is dropped in favour of `education_num` to avoid duplicate information. Data is then split into training and test sets.

Feature classification. Features are split into three groups:

- **Mutable:** can change within the counterfactual framework.
- **Searchable:** DiCE may vary them during generation.
- **Recommendable:** allowed to appear in the final action plan.

```
RECOMMENDABLE_FEATURES = [
    "education_num",
    "occupation",
    "workclass",
    "hours_per_week",
]
```

A feature that can be changed mathematically is not automatically appropriate as a real-world recommendation. For example, marital status may be searchable but is not recommendable.

4. Module 2: Predictive Modelling (predictor.py)

Logistic Regression, Random Forest and XGBoost are trained and evaluated on the held-out test split using AUC, accuracy, precision, recall and F1. The model with the highest AUC is selected:

```
scoreboard.sort(key=lambda s: s.auc, reverse=True)
best_name = scoreboard[0].name
best_model = trained_models[best_name]
```

The decision threshold is 0.50. For test-case selection, the predictor can pick a candidate within a configured probability range just below the threshold, since such a case is closer to the boundary and typically needs fewer changes to flip.

5. Module 3: Causal Structure Discovery (causal_scm.py)

5.1 PC Algorithm

The PC Algorithm is a constraint-based method that uses conditional-independence tests to remove unsupported edges. The graph is learned from data rather than defined manually, and the income target is included so its relationships can be examined directly.

```
X = loader_result.X_train.copy()
X[TARGET_FEATURE] = np.asarray(loader_result.y_train)
feature_names = list(X.columns)
```

Large datasets are subsampled for discovery. Supported CI tests are KCI, Chi-square and Fisher-Z; the current configuration uses KCI. If no edges are found at one alpha level, the next in the schedule is tried.

```
scm:
  ci_test: kci
  n_bins: 10
  max_sample_rows: 500
  random_state: 42
  alpha_schedule: [0.05, 0.2, 0.5]
```

5.2 Graph construction and eligibility

The graph is stored as undirected and directed adjacency matrices, with directed relationships represented in NetworkX. Features connected to income are found by graph reachability. Two masks are built:

- `mutable_only_mask`: variables defined as mutable.
- `bool_mask`: variables that are both mutable and connected to income in the undirected graph. This is the main mask restricting the counterfactual search.

```
bool_mask[idx] = is_mutable and (idx in reachable_to_income)
```

5.3 Interpretation

Graph connectivity does not prove an identified causal effect. The discovered graph is used as a structural constraint, not as evidence of causation.

5.4 Cycle handling

Discovered directed graphs can contain cycles (e.g. Education → Working Hours → Occupation → Education), which prevent topological ordering. The framework copies the graph and repeatedly breaks the shortest cycle by removing one edge until the copy is a DAG:

```
dag = graph.copy()
cycle_breaks = []
while not nx.is_directed_acyclic_graph(dag):
   cycles = list(nx.simple_cycles(dag))
   if not cycles:
      break
   cycle = min([tuple(c) for c in cycles], key=lambda c: (len(c), c))
   edge = (cycle[-1], cycle[0])
```

```
    if dag.has_edge(*edge):
        dag.remove_edge(*edge)
        cycle_breaks.append(edge)
```

The original graph is left unchanged and still used for eligibility checks. Removed edges are recorded in `cycle_breaks` for reporting.

6. Module 4: Counterfactual Generation (counterfactual.py)

6.1 DiCE setup and generation

```
data_interface = dice_ml.Data(
    dataframe=combined_df,
    continuous_features=all_continuous_features,
    outcome_name="income",
)
model_interface = dice_ml.Model(
    model=predictor_result.best_model, backend="sklearn"
)
explainer = Dice(data_interface, model_interface, method="random")

dice_result = explainer.generate_counterfactuals(
    query_instance,
    total_CFs=total_cfs,
    desired_class="opposite",
    features_to_vary=search_features,
)
```

`search_features` is the intersection of the searchable features and the SCM eligibility mask, so DiCE cannot freely change every input.

6.2 Validation

Each candidate passes two checks; failures are counted in `rejection_counts`.

- **SCM validation:** all changed features must satisfy the structural eligibility conditions.
- **Realism validation:** changes must satisfy domain constraints, e.g. education years cannot decrease, education changes have a maximum delta and range, working hours have numeric limits, and some categorical destinations are excluded.

6.3 Feasibility and ranking

Candidates are scored on normalized proximity, number of changed features, causal distance to income and recommendability:

```
proximity = float(np.mean(normalized_terms))
score = proximity + SPARSITY_PENALTY_WEIGHT * len(changes)
```

The final ranking adds causal-distance and recommendability penalties.

6.4 Iterative search

If valid candidates exist but none reaches the effective threshold, the candidate pool is enlarged and the search repeats.

```
counterfactual:
  max_iterations: 4
  initial_total_cfs: 5
  cfs_growth_factor: 2
  sparsity_penalty_weight: 0.5
  causal_distance_weight: 0.25
  target_margin: 0.05
```

With a threshold of 0.50 and a margin of 0.05, the effective target probability is **0.55 or higher**:

```
flipped_this_iteration = [
    c for c in ranked_this_iteration
    if c.predicted_probability >= effective_threshold
]
```

7. Module 5: Evidence Retrieval and O*NET (rag_retrieval.py)

Counterfactuals describe model-level changes but offer no practical resources, so this module retrieves relevant ones.

- **Corpus and embeddings:** resources are loaded from `resources.json` and embedded with `sentence-transformers/all-MiniLM-L6-v2` (TF-IDF fallback if unavailable). Vectors are normalized and indexed with an inner-product FAISS index.
- **Feature-aware queries:** education changes query training, certification and apprenticeships; occupation changes query career exploration and job resources; working-hours changes query practical employment options. The `applies_to` metadata prioritizes relevant resources, and non-recommendable changes are skipped.
- **O*NET (optional):** with an API key, an occupation change can retrieve the occupation code, title, description and Bright Outlook information. Without a key, the local corpus is used.

8. Module 6: Action-Plan Synthesis (llm_synthesis.py)

Three backends are available: a deterministic template, a local Ollama LLM, and the OpenAI API.

Prompt. The LLM receives the original probability, validated counterfactual changes, retrieved resource descriptions, evidence IDs and verified URLs, and is instructed not to invent resources, URLs, organizations, feature changes or probabilities.

Grounding validation. Generated URLs and evidence IDs are checked against the retrieved evidence; evidence-reference ranges and certain unrealistic workclass phrases are also checked.

```
verified_urls = {r.url for r in verified_resources if r.url}
for url in extracted_urls:
    if url not in verified_urls:
        return False
```

If validation fails or the response cannot be parsed, the deterministic template is used.

Three-phase plan.

Phase	Timeframe	Content
1	Weeks 1–2	Immediate steps and resource exploration

Phase	Timeframe	Content
2	Months 1–6	Training, qualification, skill development
3	After earlier phases	Summary of the change and projected probability

The model and Python code determine the counterfactual values and projected probability. The LLM only turns retrieved evidence into readable steps and never sets or alters numerical values.

9. Module 7: Evaluation and Baselines (evaluator.py)

9.1 Metrics

Metric	Meaning
Validity rate	Share of cases with a valid counterfactual
Average proximity	Mean normalized difference between original and modified features
Average sparsity	Mean number of changed features
Probability gain	Increase in favourable-outcome probability
Median probability gain	Median improvement across successful cases
Actionability rate	Share of successful cases with recommendable changes
Average causal distance	Mean graph distance of selected changes from the target

Case selection can be boundary-based or random (seedable for reproducibility).

9.2 SCM ranking ablation

Proximity-only ranking is compared with SCM distance-aware ranking to test whether graph distance changes the selected counterfactuals and their quality.

9.3 Baselines

The proposed pipeline is compared with Vanilla DiCE, post-hoc-filtered DiCE, the Wachter baseline and the Ustun baseline, on success rate, recommendability, proximity, sparsity, probability gain and constraint violations.

9.4 Outputs

Results are saved to `outputs/evaluation_case_results.csv` (per-case) and `outputs/evaluation_report.json` (aggregate). These files should be the source of all quantitative results in the paper.

10. Module 8: Pipeline Orchestration (main.py)

`main.py` runs seven timed stages: (1) load and preprocess data, (2) train and select the model, (3) discover causal structure, (4) generate and validate counterfactuals, (5) retrieve evidence, (6) generate

the action plan, (7) evaluate. It supports configuring the backend, case count, selection strategy and baseline comparison, and runs independently of the Streamlit app.

```
python main.py --backend template --eval-cases 10 --selection-mode random
python main.py --compare-baselines --compare-cases 5 --eval-cases 5
```

11. Streamlit Application (app.py)

The app uses the same model, causal graph and resource index as the main pipeline. Users enter a profile through dropdown and numeric fields, and choose the backend, iteration settings and initial DiCE candidate count. On "Run XAI analysis", the profile is encoded to the model's schema and the favourable-outcome probability is compared with the threshold; if it is below, counterfactual generation starts.

Four result sections are shown: (1) prediction probability and decision, (2) counterfactual changes and projected probability, (3) retrieved evidence and optional O*NET information, (4) the structured action plan. The interface notes that counterfactual probabilities are model estimates, not guaranteed real-world outcomes.

12. Limitations and Methodological Considerations

1. **Causal interpretation.** PC discovers statistical structure from CI tests, and eligibility relies on undirected connectivity to income. This is not proof of causal effects. Validation with alternative CI tests and domain knowledge is needed.
2. **Counterfactual realism.** Despite domain and graph constraints, counterfactuals are model-level alternatives; real-world feasibility must be assessed separately.
3. **LLM grounding.** The validator checks URLs, evidence IDs and selected phrases only. It does not confirm that every statement is semantically supported by the evidence; entailment or factual-consistency evaluation could be added.
4. **Fairness.** The dataset includes race and sex, so a fairness assessment across demographic groups is required.
5. **Experimental robustness.** Further experiments are needed on random seeds, alternative CI tests, alpha sensitivity, candidate-generation budgets, larger evaluation samples and baseline performance differences before making strong claims.

13. Points for Guide's Feedback

1. **Causal discovery.** Is PC with KCI appropriate for the mixed numerical and categorical variables in Adult Income? Would a different mixed-data CI approach or extra sensitivity analysis be better?
2. **SCM integration.** Is undirected connectivity to income a reasonable eligibility constraint, or should it be complemented by further causal validation?
3. **Cycle handling.** Is using a temporary DAG for ordering, while keeping the original graph for validation, acceptable?
4. **Counterfactual constraints.** Are the feasibility, sparsity, causal-distance and domain constraints sufficient? What other realism or actionability measures should be added?
5. **Baselines.** Are the chosen baselines sufficient, or should more comparison methods be included?

6. **LLM grounding.** Would semantic entailment or factual-consistency evaluation improve reliability?
7. **Contribution.** Is the combination of causal eligibility, constrained counterfactuals, evidence retrieval and action-plan synthesis a sufficiently distinct contribution, and which component should the paper emphasize?

14. Proposed Next Steps

1. Run repeated multi-case evaluation with consistent settings.
2. Strengthen LLM grounding and factual-consistency evaluation.
3. Conduct fairness analysis across demographic groups.
4. Improve quantitative comparison against baselines.
5. Consolidate implementation and results for the final paper.

15. Conclusion

The framework integrates prediction, causal structure discovery, constrained counterfactual generation, evidence retrieval and LLM-based action-plan synthesis in a modular pipeline with an interactive Streamlit app. Causal interpretation, counterfactual feasibility, retrieval effectiveness, fairness and evaluation robustness still need further investigation.